EVENT TRACKING · 101

Client vs proxy vs *server.*

WHERE TO FIRE EACH EVENT
AND WHY IT MATTERS FOR YOUR DATA.

DATALYZE
JOINDATALYZE.COM

Web loses *one in four* events. Mobile barely loses any.

WEB

20–30%

Ad blockers, ITP, tracking protection, corporate firewalls, analytics-domain blockers.

If you're tracking purely client-side on web, assume one in four events never makes it to your vendor.

MOBILE

5–10%

No ad blocker inside the Swiggy app. The SDK is compiled into the binary.

Most loss is crashes before the event flushes, uninstalls before queued events sync, or genuinely offline users.

Every event you track gets fired from *one of three places.*

01 · Easiest

Client

Runs on the user's device. Auto-captures behavior. Easy to block.

`user device → vendor API`

02 · Best of both

Proxy

Same client SDK, pointed at your own subdomain. Looks first-party. Not blocked.

`user device → your proxy → vendor API`

03 · Bulletproof

Server

Fired from your backend. Tied to your database. Can't see the browser.

`your backend → vendor API`

01 · CLIENT-SIDE

Runs on the user's device. Easiest to set up. *Easiest to block.*

THE PATH

`User device` → `Vendor API`

Event fires in browser JS or mobile SDK → goes **directly** to Mixpanel, GA4, Amplitude. No intermediary.

THE TRADE

- **Captures behavior automatically** — clicks, scrolls, page views, all of it for free.
- **Ad blockers eat 15–40%** of events before they leave the device.
- **API keys are exposed** in the page source.
- **You're stuck** with whatever the vendor accepts.

Same client SDK, pointed at *your own subdomain.*

THE PATH

`User device` → `track.yourdomain.com` → `Vendor API`

Your proxy receives the event, then forwards it to the vendor. The browser sees **a first-party request** — not `api.mixpanel.com`.

THE TRADE

- **Ad blockers don't catch it** — it looks like your own domain.
- You can **enrich, filter, or rewrite** events before they leave.
- You **own the data** going through.
- You run the infrastructure — one more service to operate.

Fired from your backend, not the user's device. *Bulletproof.*

THE PATH

`Your backend` → `Vendor API`

User signs up → your auth service calls Mixpanel's HTTP API directly with `signup_completed`. The event is tied to the database write, not a button click.

THE TRADE

- **Can't be blocked.** Can't be spoofed.
- **Ties directly to your database truth** — Stripe, billing, the source row in Postgres.
- **Can't see client behavior** — scrolls, hovers, time on page, none of it.
- Every property you want, you have to pass **explicitly.**

THE ONE-LINE DIFFERENCE

Client is what the user *did*.

Server is what *actually happened*.

Proxy is client tracking, *unblocked*.

All this context comes with every event. *Server has none of it.*

DEVICE & BROWSER

User agent, browser name & version, OS, device model, screen resolution, viewport, language, timezone.

GEO

IP-derived country, region, city.

PAGE CONTEXT

URL, title, path, query, hash, **referrer**, previous page.

SESSION

session_id, session duration, time on page, page-view sequence within the session.

MARKETING ATTRIBUTION

UTM source, medium, campaign, term, content — captured automatically from the URL.

ANONYMOUS IDENTITY

anonymous_id / distinct_id generated before a user logs in. The entire pre-signup funnel.

ENGAGEMENT SIGNALS

Scroll depth, time on page, rage clicks, dead clicks, mouse movement.

PERFORMANCE

Page load time, LCP, CLS, time to first byte.

MOBILE-SPECIFIC

App version, build number, network type, carrier, install referrer, push notification context.

THE BLIND SPOT

Your backend only knows what's in the *request body and headers.*

It doesn't know whether the user *scrolled to the bottom of your pricing page* before clicking "Talk to Sales."

It doesn't know what *UTM brought them in three days ago.*

> **Go pure server-side and you have to pass every piece of this context manually with every event.** Most teams forget half of it — and then build dashboards on top of the half that's left.

THE KILLER ARGUMENT

Proxy + server is the *right combo.*

PROXY GIVES YOU

All the rich behavioral context **for free.** Sessions, attribution, anonymous identity, engagement, performance.

SERVER GIVES YOU

Source-of-truth business events. **Signups, payments, subscriptions, refunds.** Things you'd reconcile against Stripe.

Together they give you behavior + truth. Apart, you're missing one or the other.

Send through the proxy.

BEHAVIOR & INTENT

- **Page views, screen views**
- Clicks, hovers, scroll depth, video plays
- Form started, field abandoned, validation errors
- Feature engagement — settings opened, search used, empty state explored

CONTEXT THE SERVER CAN'T SEE

- **Anonymous pre-signup behavior** — the big one
- Frontend errors and performance metrics
- A/B test exposure events
- Session start, session end

Send from your server.

IDENTITY & AUTH

- `signup_completed` — after the DB write, not the click
- `login_success`
- `password_reset`
- `email_verified`

MONEY

- `subscription_created · upgraded · cancelled`
- `payment_succeeded · failed`
- `refund_issued`
- `trial_started · converted · expired`

FULFILLMENT & OPS

- `order_placed · order_fulfilled`

DRIVEN BY EXTERNAL SYSTEMS

- `Webhook-driven events` (Stripe, Calendly, integrations)
- `Cron-fired events` (weekly_digest_sent, churn_predicted)

> **Rule of thumb:** anything you'd reconcile against Stripe, your database, or your billing system → server-side.

THE SIMPLE RULE

If the truth lives in your *database,*
fire it from your *server.*

If the truth lives in the user's
session, fire it through the *proxy.*